

ABSTRACT

In this work, we aim at building a bridge from coarse behavioral data to an effective, quick-response, and robust behavioral model for online identity theft detection. We concentrate on this issue in online social networks (OSNs) where users usually have composite behavioral records, consisting of multidimensional low-quality data, e.g., offline check-ins and online user-generated content (UGC). As an insightful result, we validate that there is a complementary effect among different dimensions of records for modeling users' behavioral patterns. To deeply exploit such a complementary effect, we propose a *joint* (instead of *fused*) model to capture both online and offline features of a user's composite behavior. We evaluate the proposed joint model by comparing it with typical models and their fused model on two real-world datasets: Foursquare and Yelp. The experimental results show that our model outperforms the existing ones, with the area under the receiver operating characteristic curve (AUC) values 0.956 in Foursquare and 0.947 in Yelp, respectively. Particularly, the *recall* (true positive rate) can reach up to 65.3% in Foursquare and 72.2% in Yelp with the corresponding *disturbance rate* (false-positive rate) below 1%. It is worth mentioning that these performances can be achieved by examining only one composite behavior, which guarantees the low response latency of our method. This study would give the cybersecurity community new insights into whether and how real-time online identity authentication can be improved via modeling users' composite behavioral patterns.

INDEX

S.NO.	CONTENTS	PAGE NO.
1.	INTRODUCTION	01-05
2.	LITERATURE SURVEY	06-09
3.	SYSTEM ANALYSIS	10-15
4.	SYSTEM DESIGN	16-24
5.	SOFTWARE ENVIRONMENT	25-45
6.	SYSTEM TESTING	46-49
7.	SCREEN SHOTS	50-58
8.	CONCLUSION	59-60
	REFERENCES	61-63

CHAPTER-I

INTRODUCTION

1. INTRODUCTION

WITH the rapid development of the Internet, more and more affairs, e.g., mailing [1], health caring [2], shopping [3], booking hotels, and purchasing tickets, are handled online [4]. Meanwhile, the Internet also brings sundry potential risks of invasions, such as losing financial information [5], identity theft [6], and privacy leakage [3]. Online accounts serve as the agents of users in the cyber world. Online identity theft is a typical online crime which is the deliberate use of another person's account [7], usually as a method to gain a financial advantage or obtain credit and other benefits in another person's name. As a matter of fact, compromised accounts are usually the portals of most cybercrimes [1], such as blackmail [5], fraud [8], and spam [9], [10]. Thus, identity theft detection is essential to guarantee users' security in the cyber world.

Traditional identity authentication methods are mostly based on access control schemes, e.g., passwords and tokens [11], [12]. But users have some overheads in managing dedicated passwords or tokens. Accordingly, the biometric identification [13]–[15] is delicately introduced to start the era of password-free. However, some disadvantages make these access control schemes still incapable of being effective in real-time online services [16], [17].

- 1) They are not nonintrusive. Users have to spend extra time in the authentication.
- 2) They are not continuous. The defending system will fail to take further protection once the access control is broken.

Behavior-based suspicious account detection [16], [18], [19] is a highly anticipated solution to pursue a nonintrusive and continuous identity authentication for online services. It depends on capturing users' suspicious behavior patterns to discriminate the suspicious accounts. The problem can be divided into two categories: fake/sybil account detection [20] and compromised account detection [21]. The fake/Sybil account's behaviors usually do not conform to the behavioral pattern of the majority. Meantime, the compromised account usually behaves in a pattern that does not conform to its previous one, even behaves like fake/sybil accounts. It can be solved by capturing mutations of users' behavioral patterns. Comparing with detecting compromised accounts, detecting fake/sybil accounts is relatively easy since the latter's behaviors are generally more detectable than the former's. It has been extensively studied and can be realized by various population-level approaches, e.g., clustering [22], [23], classification [5], [24]–[26] and statistical or empirical rules [8], [27], [28]. Thus, we only focus on the compromised account detection, commonly called identity theft detection, based on individual-level behavioral models.

Recently, researchers have proposed the individual-level identity theft detection methods by using suspicious behavior detection [9], [29]–[35]. The efficacy of these methods significantly depends on the sufficiency of behavior records. They are usually suffering from the low-quality of behavior records due to data collecting limitations or some privacy issues [3]. In particular, when a method only utilizes a specific dimension of behavioral data, the efficacy damaged by poor data is possibly enlarged and the scope of application is limited. Unfortunately, many existing works just concentrate on a specific dimension of users’ behavior, such as keystroke [29], click stream [32], [36], touch-interaction [37], and user generated content (UGC) [9], [33], [34], [38].

In this project, we propose an approach to detect identity theft by using multidimensional behavioral records which are possibly insufficient in each dimension. According to such characteristics, we choose the online social network (OSN) as a typical scenario where most users’ behaviors are coarsely recorded [39]. In the Internet era, users’ behaviors are composited by offline behaviors, online behaviors, social behaviors, and perceptual/cognitive behaviors. The behavioral data can be collected in many applications, such as offline check-ins in location-based services (LBSs), online tips-posting in instant messaging services, and social relationship-making in online social services. Accordingly, we design our method based on users’ composite behaviors by these categories.

In OSNs, user behavioral data that can be used for online identity theft detection are often too low-quality or restricted to build qualified behavioral models due to the difficulty of data collection, the requirement of user privacy, and the fact that some users have a few several behavioral records. We devote ourselves to proving that a high-quality (effective, quick response, and robust) behavioral model can be obtained by integrally using multidimensional behavioral data, even though the data is extremely insufficient in each dimension.

Generally, there are two paradigms to integrate behavioral data: the fused and joint manners. Fused models are a relatively simple and straightforward kind of composite behavior models (CBMs). They first capture features in each behavior space and then make a comprehensive metric based on these features in different dimensions. With the possible complementary effect among different behavior spaces, they can act as a feasible solution for integration [7], [17]. However, the identification efficacy can be further improved, since fused models neglect potential links among different spaces of behaviors. We take an example where a person posted a picture in an OSN when he/she visited a park.

If this composite behavior is simply separated into two independent parts: he/she once posted a picture and he/she once visited a park, the difficulty in relocating him/her from a group of users is possibly increased, since there are more users satisfy these two simple conditions comparing to the original condition.

In contrast, the joint model can sufficiently exploit the correlations between behaviors in different dimensions, then increases the certainty of users' behavior patterns, which contributes to a better identification efficacy. The underlying logic for the difference between the joint and fused models can be also explained by the well-known Chain Rule for Entropy [40], which indicates that the entropy of multiple simultaneous events is no more than the sum of the entropies of each individual event, and are equal if the events are independent. It shows that the joint behavior has lower uncertainty comparing to the sum of the uncertainty in each component [41].

Therefore, to fully utilize potential information in composite behaviors for user profiling, we propose a joint model, specifically, a joint probabilistic generative model based on Bayesian networks called CBM. It offers a composition of the typical features in two different behavior spaces: check-in location in offline behavior space and UGC in online behavior space. Considering the composite behavior of a user, we assume that the generative mechanism is as follows. When a user plans to visit a venue and simultaneously post tips online, he/she subconsciously selects a specific behavioral pattern according to his/her behavioral distribution. Then, he/she comes up with a topic and a targeted venue based on the present pattern's topic and venue distributions, respectively. Finally, his/her comments are generated following the corresponding topic-word distribution. To estimate the parameters of the mentioned distributions, we adopt the collapsed Gibbs sampling [42].

Based on the joint model CBM, for each composite behavior, denoted by a triple-tuple (u, v, D) , we can calculate the chance of user u visiting venue v and posting a tip online with a set of words D . Taking into account different levels of activity of different users, we devise a relative anomalous score S_r to measure the occurrence rate of each composite behavior (u, v, D) . By these approaches, we finally realize real-time detection (i.e., judging by only one composite behavior) for identity theft suspects.

We evaluate our joint model by comparing it with three typical models and their fused model [17] on two real-world OSN datasets: Foursquare [43] and Yelp [44]. We adopt the area under the receiver operating characteristic curve (AUC) as the detection efficacy.

Particularly, the recall [true positive rate (TPR)] reaches up to 65.3% in Foursquare and 72.2% in Yelp, respectively, with the corresponding disturbance rate [false-positive rate (FPR)] below 1%, while the fused model can only achieve 60.8% and 60.4% in the same condition, respectively. Note that this performance can be achieved by examining only one composite behavior per authentication, which guarantees the low response latency of our detection method. As an insightful result, we learn that the complementary effect does exist among different dimensions of low-quality records for modeling users' behaviors.

The main contributions are summarized into three folds.

- 1) We propose a joint model, CBM, to capture both online and offline features of a user's composite behavior to fully exploit coarse behavioral data.
- 2) We devise a relative anomalous score S_r to measure the occurrence rate of each composite behavior for realizing real-time identity theft detection.
- 3) We perform experiments on two real-world datasets to demonstrate the effectiveness of CBM. The results show that our model outperforms the existing models and has the low response latency.

The rest of this project is organized as follows. We give an overview of our solution in Section II. Then, we present our method in Section III, and make the validation in Section IV. We provide a literature review in Section V. Finally, we draw conclusions in Section VI.

CHAPTER-II

LITERATURE SURVEY

2. LITERATURE SURVEY

To prevent and detect identity theft in online services, developers have designed various authentication methods to identify a user's identity. Traditional password-based (username-password) authentication methods are still widely used. But the password is easy to leak, forget, and copy. Then, an authentication method adopted a physical token instead of a password [52], but it was easy to lose the token. Since the biological characteristics are hard to copy or change, more and more applications turn to utilize biometric identification technologies, such as fingerprint, face, iris, and speech recognition which are stable and not varying with time, for authentication [13], [14]. Sitova et al. [53] introduced hand movement, orientation, and grasp (HMOG), a set of behavioral features to continuously authenticate smartphone users. Rajoub and Zwiggelaar [15] used thermal imaging to monitor the periorbital region's thermal variations and test whether it can offer a discriminative signature for detecting deception. However, these biometric technologies usually require expensive hardware devices which makes it inconvenient and difficult to popularize. Another drawback for these methods above is that they are redundant steps, which require users to spend extra time passing identification. Besides, they are all disposable identification measures. Once a criminal breaks through the wall, the defending system will fail to take further protection. On the contrary, when attackers facing with a continuous authentication system, they have to spend a prolonged time fooling the system. Increasingly, researchers and security experts realized that they cannot ensure users' security just by building higher and stronger digital walls around everything [16]. Thus, it is urgent to establish a non-intrusive and continuous authentication system. Recently, researchers found that users' behavior can identify their identity and judge their personality. A study on three months of credit card records for 1.1 million people showed that four spatiotemporal points are enough to uniquely reidentify 90% of individuals [3]. Research studies found that computers outpacing humans in personality judgment presented significant opportunities and challenges in the areas of psychological assessment, marketing, and privacy [54]. Abouelenien et al. [30] explored a multimodal deception detection approach that relied on a novel dataset of 149 multimodal recordings, and integrated multiple physiological, linguistic, and thermal features. These works indicated that users' behavior patterns can represent their identities. Many studies turn to utilize users' behavior patterns for identifications. Behavior-based methods were born at the right moment, which plays important roles in a wide range of tasks including preventing and detecting identity theft. Typically, behavior-based user identification includes two phases: user profiling and user identifying. User profiling is a process to characterize a user with his/her history behavioral data.

Some works focus on statistical characteristics, such as the mean, variance, median, or frequency of a variable, to establish the user profile. Naini et al. [55] studied the task of identifying the users by matching the histograms of their data in the anonymous dataset with the histograms from the original dataset. But it mainly relied on experts' experience since different cases usually have different characteristics. Egele et al. [7] proposed a behavior-based method to identify compromises of individual high-profile accounts. However, it required high-profile accounts which were difficult to obtain. Other researchers discovered other features, such as tracing patterns, topic and spatial distributions, to describe user identity. Ruan et al. [32] conducted a study on online user behavior by collecting and analyzing user clickstreams of a well-known OSN. Lesaege et al. [31] developed a topic model extending the LDA to identify the active users. Viswanath et al. [56] presented a technique based on principal component analysis (PCA) that accurately modeled the "like" behavior of normal users in Facebook and identified significant deviations from it as anomalous behaviors. Zaeem et al. [33] proposed an approach that involved the novel collection of online news stories and reports on the topic of identity theft. Lichman and Smyth [48] proposed MKDE model to accurately characterize and predict the spatial pattern of an individual's events. Tsikerdekis and Zeadally [57] presented a detection method based on nonverbal behavior for identity deception, which can be applied to many types of social media. These methods above mainly concentrated on a specific dimension of the composite behavior and seldom thought about utilizing multidimensional behavior data. Sekara et al. [58] explored the complex interaction between social and geospatial behavior and demonstrated that social behavior can be predicted with high precision. It indicated that composite behavior features can identify one's identity. Yin et al. [42] proposed a probabilistic generative model combining the use of spatiotemporal data and semantic information to predict user's behavior. Nilizadeh et al. [49] presented POISED, a system that leverages the differences in propagation between benign and malicious messages on social networks to identify spam and other unwanted content. These studies implied that composite behavior features are possibly helpful for user identification. User identification is a process to match the same user in two datasets or distinguish anomalous users/behaviors. User identifying can be applied to a variety of tasks, such as detecting anomalous users or match users across different data sources. Mazzawi et al. [59] presented a novel approach for detecting malicious user activity in databases by checking user's self-consistency and global consistency. Shabtai et al. [60] presented a behavior-based anomaly detection system for detecting meaningful deviations in a mobile application's network to protect mobile device users and cellular infrastructure companies from malicious applications.

Lee and Kim [34] proposed a suspicious URL detection system for Twitter to detect anomalous behavior. Thomas et al. [9] leveraged Monarch’s feature collection infrastructure to study distinctions among 11 million URLs drawn from email and Twitter. These works mainly detected population-level anomalous behaviors which indicated a strong difference to other behaviors. They did not take into account the coherence of a user’s behavioral records since their works implied that anomalous accounts were created by criminals (i.e., fake accounts). Cao et al. [23] designed and implemented a malicious account detection system for detecting both fake and compromised real user accounts. Zhou et al. [61] designed an FRUI algorithm to match users among multiple OSNs. Hao et al. [62] proposed a novel framework for user identification in cyberphysical space. Most of the existing related works mainly considered specific dimensions of users’ behavior. Sufficient high-quality data are necessary for these works. In this study, we aim to build high-performance behavioral models based on low-quality behavioral data by integrating different dimensions of behavioral records that are usually too sparse to support qualified models. The most relevant work to our study is [17], where different dimensions of behavioral records are fused to build a CBM for detecting identity theft under a special pattern.

CHAPTER-III

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

Sitova et al. [53] introduced hand movement, orientation, and grasp (HMOG), a set of behavioral features to continuously authenticate smartphone users. Rajoub and Zwiggelaar [15] used thermal imaging to monitor the periorbital region's thermal variations and test whether it can offer a discriminative signature for detecting deception. However, these biometric technologies usually require expensive hardware devices which makes it inconvenient and difficult to popularize.

Abouelenien et al. [30] explored a multimodal deception detection approach that relied on a novel dataset of 149 multimodal recordings, and integrated multiple physiological, linguistic, and thermal features. These works indicated that users' behavior patterns can represent their identities. Many studies turn to utilize users' behavior patterns for identifications. Behavior-based methods were born at the right

moment, which plays important roles in a wide range of tasks including preventing and detecting identity theft. Typically, behavior-based user identification includes two phases: user profiling and user identifying.

User profiling is a process to characterize a user with his/her history behavioral data. Some works focus on statistical characteristics, such as the mean, variance, median, or frequency of a variable, to establish the user profile. Naini et al. [55] studied the task of identifying the users by matching the histograms of their data in the anonymous dataset with the histograms from the original dataset. But it mainly relied on experts' experience since different cases usually have different characteristics.

Egele et al. [7] proposed a behavior-based method to identify compromises of individual high-profile accounts. However, it required high-profile accounts which were difficult to obtain.

Other researchers discovered other features, such as tracing patterns, topic and spatial distributions, to describe user identity. Ruan et al. [32] conducted a study on online user behavior by collecting and analyzing user clickstreams of a well-known OSN. Lesaege et al. [31] developed a topic model extending the LDA to identify the active users. Viswanath et al. [56] presented a technique based on principal component analysis (PCA) that accurately modeled the "like" behavior of normal users in Facebook and identified significant deviations from it as anomalous behaviors. Zaeem et al. [33] proposed an approach that involved the novel collection of online news stories and reports on the topic of identity theft. Lichman and Smyth [48] proposed MKDE model to accurately characterize and predict the spatial pattern of an individual's events.

Tsikerdekis and Zeadally [57] presented a detection method based on nonverbal behavior for identity deception, which can be applied to many types of social media. These methods above mainly concentrated on a specific dimension of the composite behavior and seldom thought about utilizing multidimensional behavior data. Sekara et al. [58] explored the complex interaction between social and geospatial behavior and demonstrated that social behavior can be predicted with high precision. It indicated that composite behavior features can identify one's identity.

Yin et al. [42] proposed a probabilistic generative model combining the use of spatiotemporal data and semantic information to predict user's behavior. Nilizadeh et al. [49] presented POISED, a system that leverages the differences in propagation between benign and malicious messages on social networks to identify spam and other unwanted content. These studies implied that composite behavior features are possibly helpful for user identification.

DISADVANTAGES OF EXISTING SYSTEM:

- 1) LDA model performs poorly in both datasets which may indicate its performance is strongly sensitive to the data quality.
- 2) CF-KDE and LDA model performs not well in Yelp dataset comparing to Foursquare dataset, but the fused model [17] observes a surprising reversion.
- 3) The joint model based on relative anomalous score S_r outperforms the model based on logarithmic anomalous score S_l .
- 4) The joint model (i.e., JOINT-SR, the joint model in the following content of the system all refer to the joint model based on S_r) is indeed superior to the fused model.

3.2 PROPOSED SYSTEM:

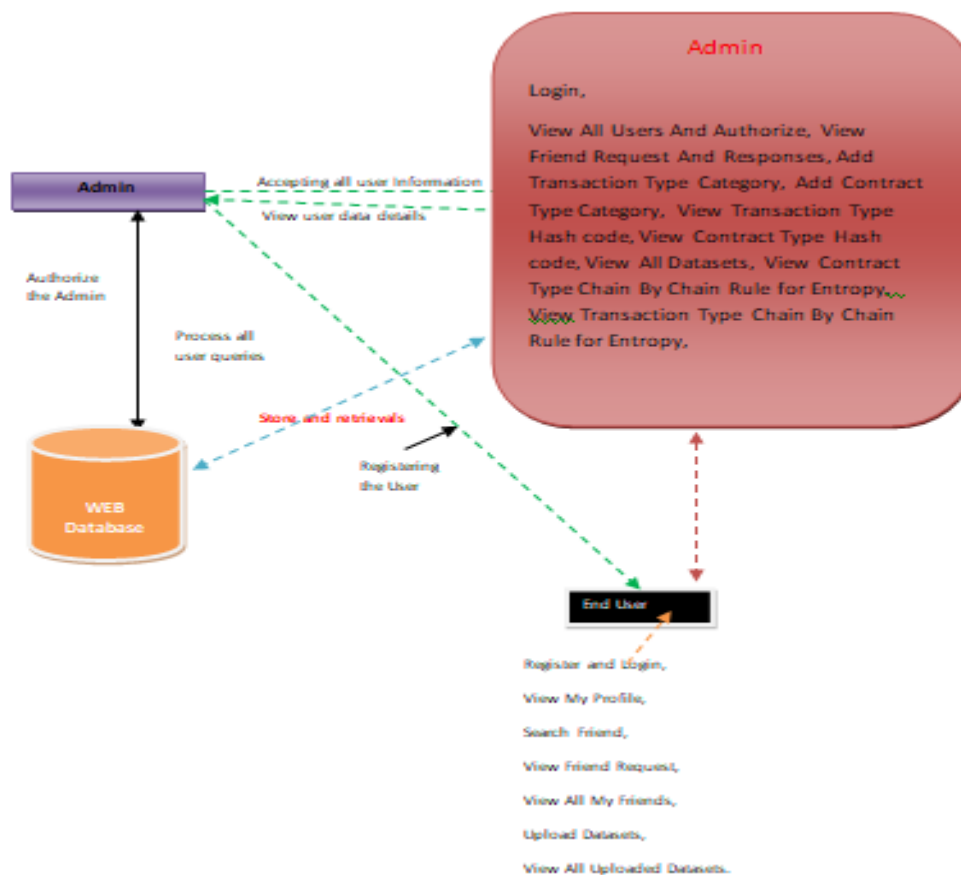
In this project, we propose an approach to detect identity theft by using multidimensional behavioral records which are possibly insufficient in each dimension. According to such characteristics, we choose the online social network (OSN) as a typical scenario where most users' behaviors are coarsely recorded [39]. In the Internet era, users' behaviors are composited by offline behaviors, online behaviors, social behaviors, and perceptual/cognitive behaviors. The behavioral data can be collected in many applications, such as offline check-ins in location-based services (LBSs), online tips-posting in instant messaging services, and social relationship-making in online social services. Accordingly, we design our method based on users' composite behaviors by these categories.

In OSNs, user behavioral data that can be used for online identity theft detection are often too low-quality or restricted to build qualified behavioral models due to the difficulty of data collection, the requirement of user privacy, and the fact that some users have a few several behavioral records. We devote ourselves to proving that a high-quality (effective, quickresponse, and robust) behavioral model can be obtained by integrally using multidimensional behavioral data, even though the data is extremely insufficient in each dimension.

ADVANTAGES OF PROPOSED SYSTEM:

- 1) We propose a joint model, CBM, to capture both online and offline features of a user's composite behavior to fully exploit coarse behavioral data.
- 2) We devise a relative anomalous score S_r to measure the occurrence rate of each composite behavior for realizing real-time identity theft detection.
- 3) We perform experiments on two real-world datasets to demonstrate the effectiveness of CBM. The results show that our model outperforms the existing models and has the low response latency.

3.3 SYSTEM ARCHITECTURE:



3.4 HARDWARE & SOFTWARE REQUIREMENTS:

HARD REQUIRMENTS:

- System : i3 or above.
- Ram : 4 GB.
- Hard Disk : 40 GB

SOFTWARE REQUIRMENTS:

- Operating system : Windows8 or Above.
- Coding Language : python

FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-IV

SYSTEM DESIGN

4. SYSTEM DESIGN

4.1 UML DIAGRAM'S:

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

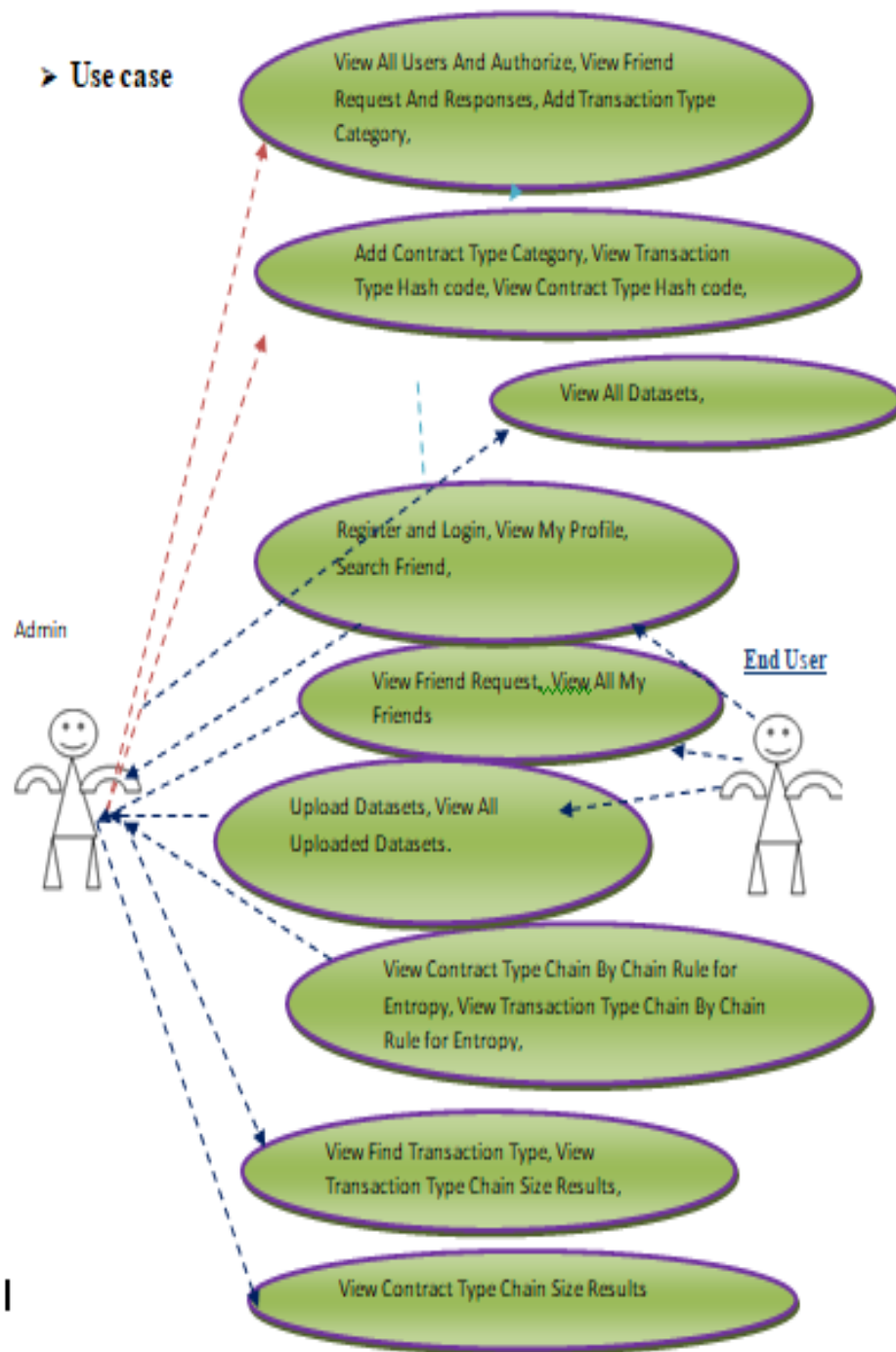
GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

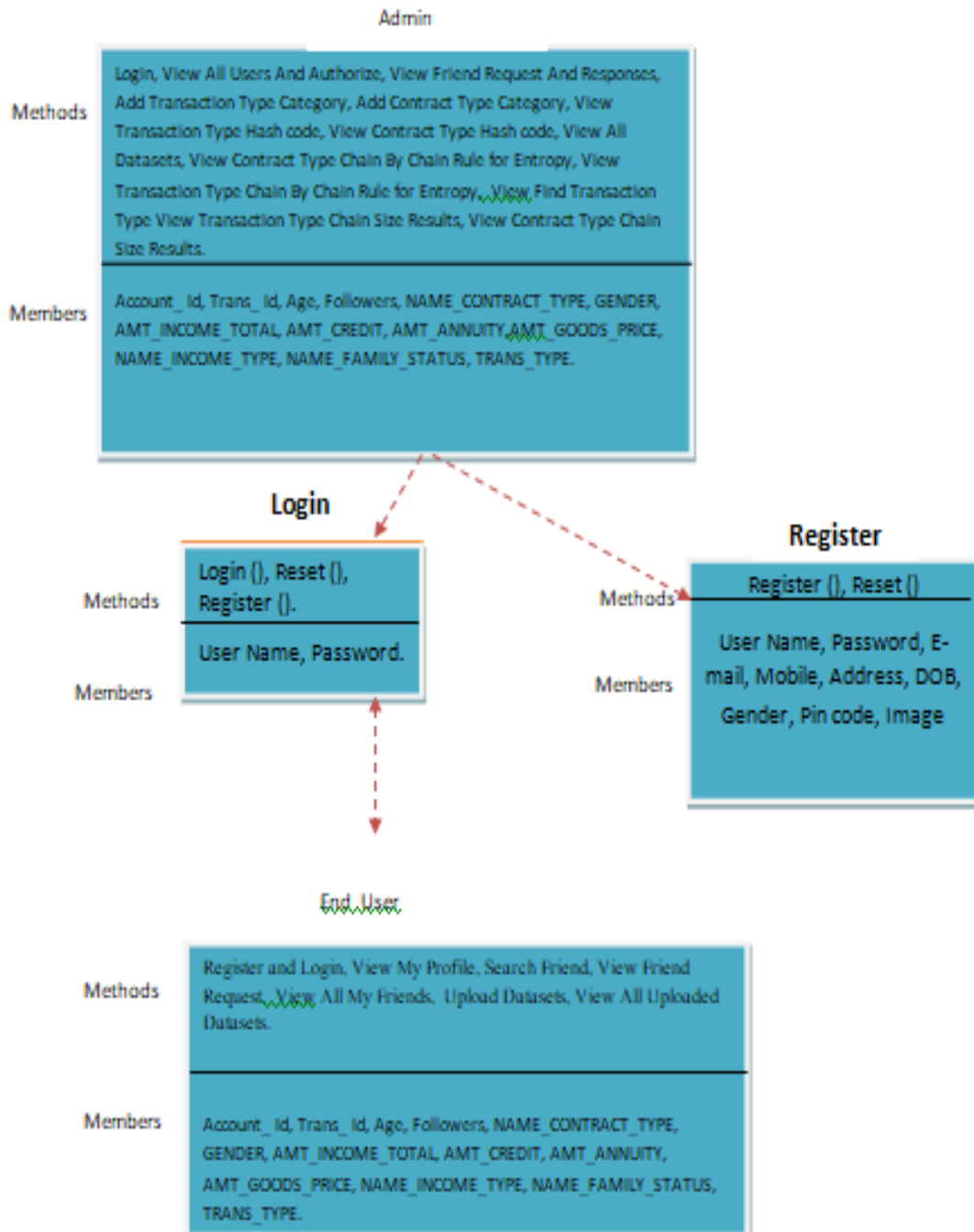
4.2 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



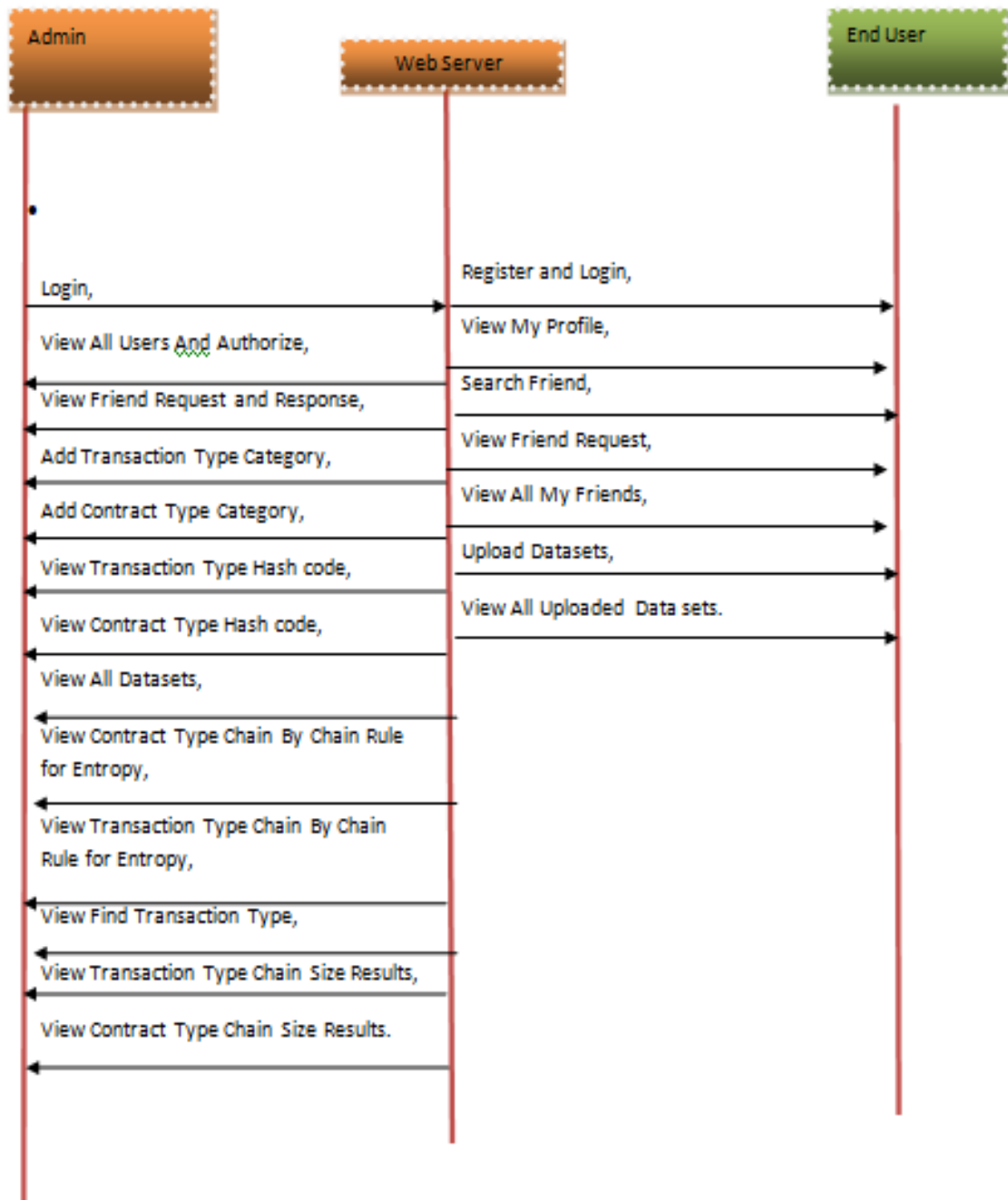
4.3 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

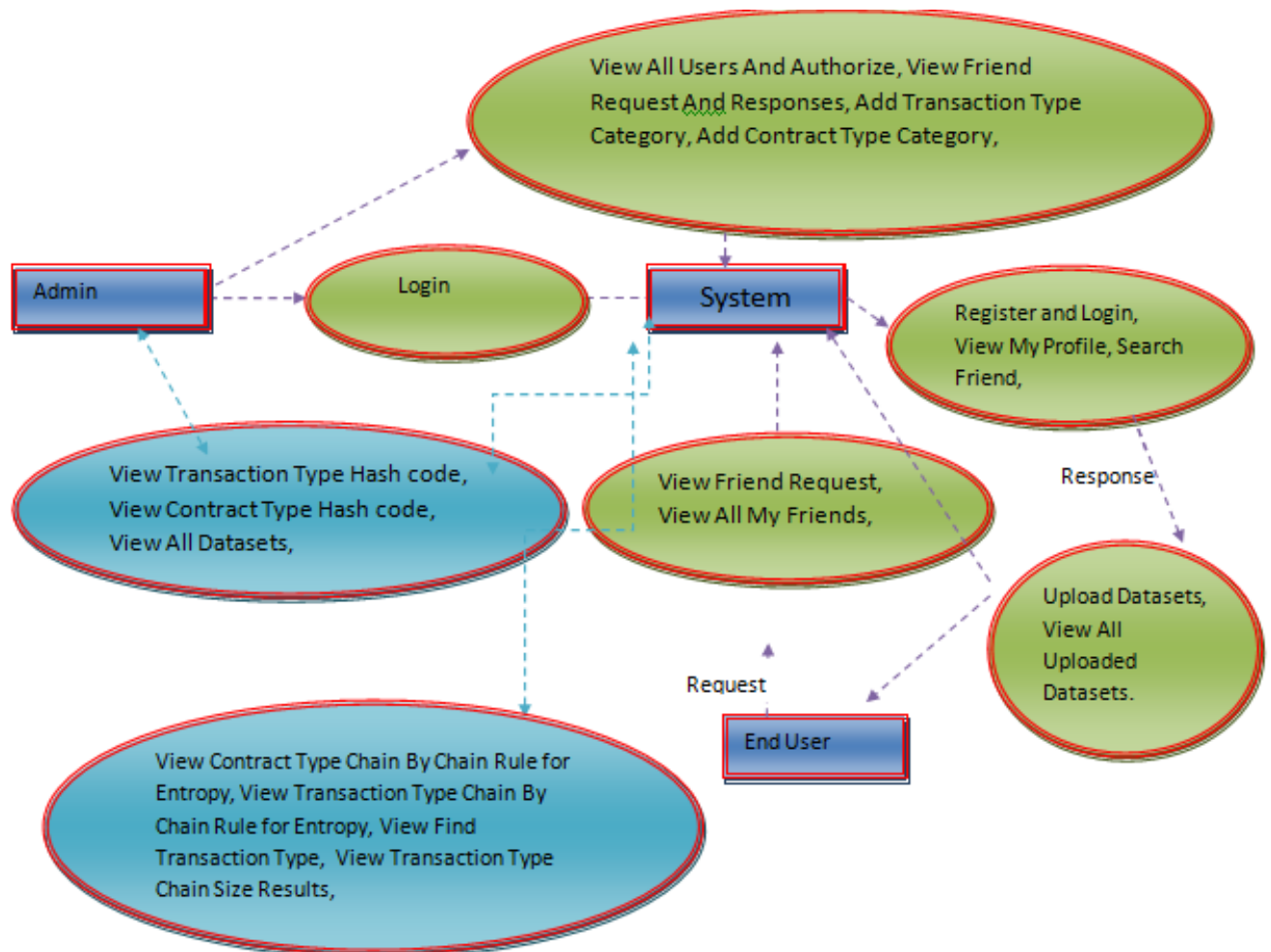


4.4 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

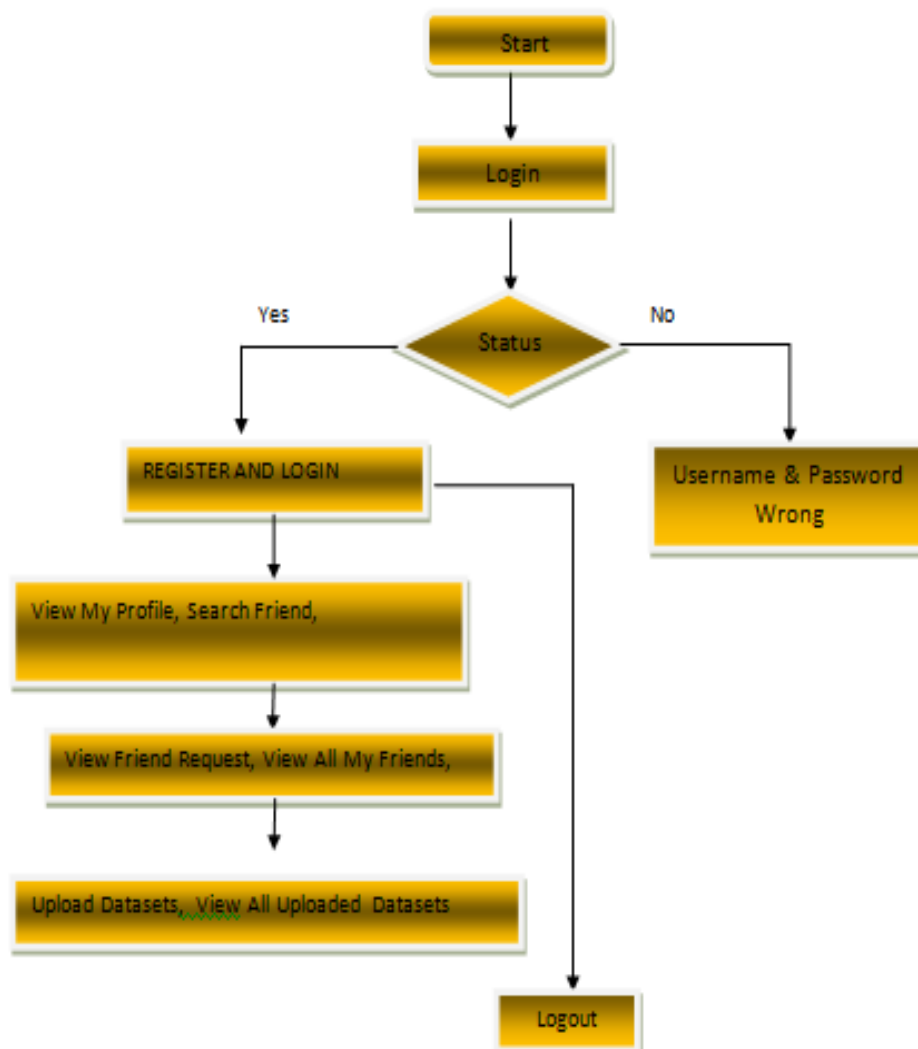


4.5 DATA FLOW DIAGRAM

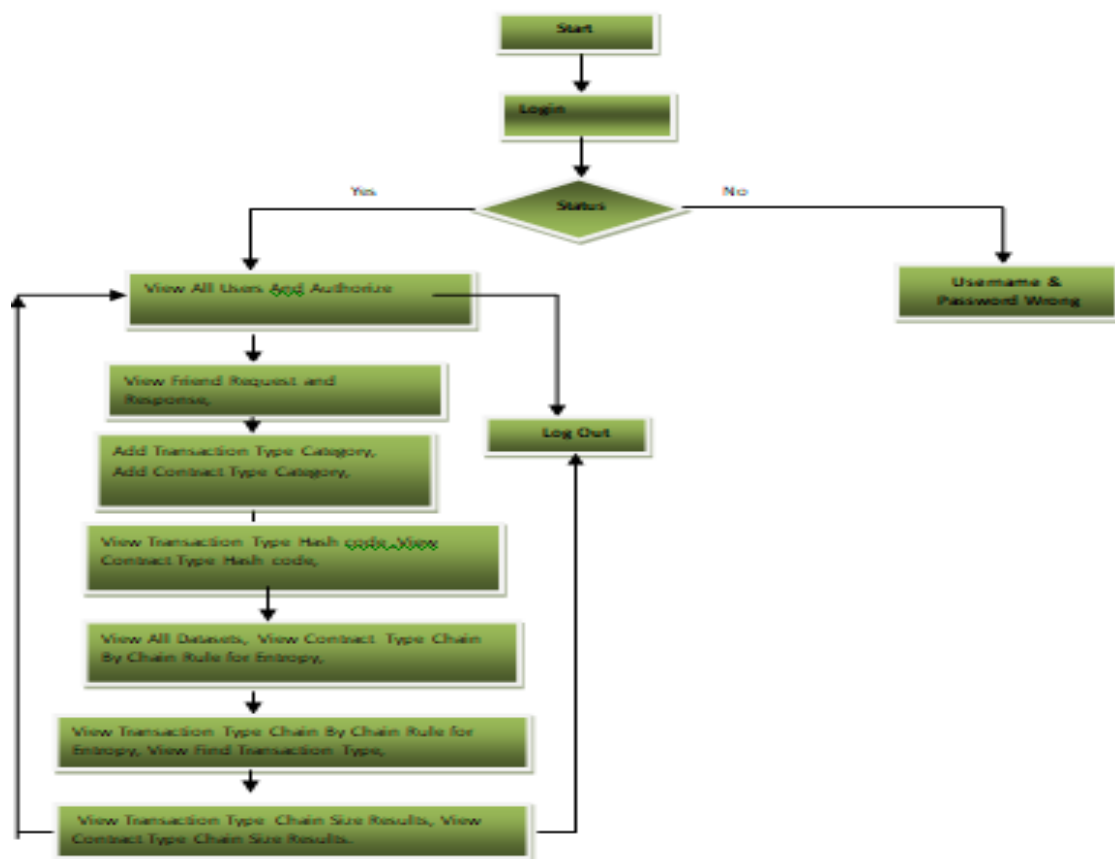


4.6 FLOW CHART DIAGRAM

➤ Flow Chart : End User



➤ Flow Chart : Admin



4.7 IMPLEMENTATION:

MODULES:

Admin

In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as

Login, View All Users And Authorize, View Friend Request And Responses, Add Transaction Type Category, Add Contract Type Category, View Transaction Type Hash code, View Contract Type Hash code, View All Datasets, View Contract Type Chain By Chain Rule for Entropy, View Transaction Type Chain By Chain Rule for Entropy, View Find Transaction Type, View Transaction Type Chain Size Results, View Contract Type Chain Size Results.

View and Authorize Users

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorizes the users.

End User

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like Register and Login, View My Profile, Search Friend, View Friend Request, View All My Friends, Upload Datasets, View All Uploaded Datasets.

CHAPTER-V

SOFTWARE ENVIRONMENT

5. SOFTWARE ENVIRONMENT

What is Python :

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQtetc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

When working with Java, you may have to create a class to print '**Hello World**'. But in Python, just a print statement will do. It is also quite **easy to learn, understand, and code**. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory**. This further aids the readability of the code.

8. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the **encapsulation of data** and functions into one.

9. Free and Open-Source

Like we said earlier, Python is **freely available**. But not only can you **download Python** for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages :

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machine learning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java DataBase Connectivity)** and **ODBC (Open DataBase Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python :-

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde&Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects.

Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML –

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term “**Machine Learning**” in 1959 and defined it as a “**Field of study that gives computers the capability to learn without being explicitly programmed**”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to Indeed, Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of **\$146,085** per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So this project deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let’s get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as **Fork Python** available Free on GeeksforGeeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans.

For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As **ML algorithms** gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps :-

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it." Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "/" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project:-

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet there. The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	 Download	Release Notes
Python 3.6.9	July 2, 2019	 Download	Release Notes
Python 3.7.3	March 25, 2019	 Download	Release Notes
Python 3.4.10	March 18, 2019	 Download	Release Notes
Python 3.5.7	March 18, 2019	 Download	Release Notes
Python 2.7.16	March 4, 2019	 Download	Release Notes
Python 3.7.2	Dec. 24, 2018	 Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files					
Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		68111671e5b2db4ae77b9ab01b0f09be	23017663	5xG
XZ compressed source tarball	Source release		d33e4aae6097051c2eca45ee3604803	17131432	5xG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.8 and later	6428b4fa7583daf1a42c8a1ce08e6	34898436	5xG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b241f	28082845	5xG
Windows help file	Windows		d63999573a2c96b2ac56c4de6b47cfd2	8131761	5xG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9b00c3c8d9ec0b9abe83184a0728a2	7504391	5xG
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0ad76d4b0b3043a583e563400	26880348	5xG
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28c31c9088b072aeb653a3bd351b4bcd2	1362904	5xG
Windows x86 embeddable zip file	Windows		9fab3b618b41879fda94133574139d8	6741626	5xG
Windows x86 executable installer	Windows		33cc902942a54446a3d8451478394789	25663848	5xG
Windows x86 web-based installer	Windows		1b670cfa5d317d82c30983ea371d87c	1324608	5xG

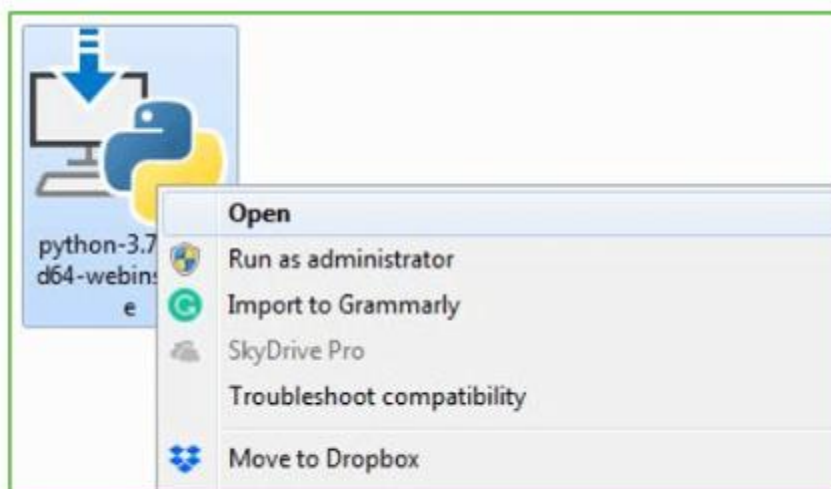
- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



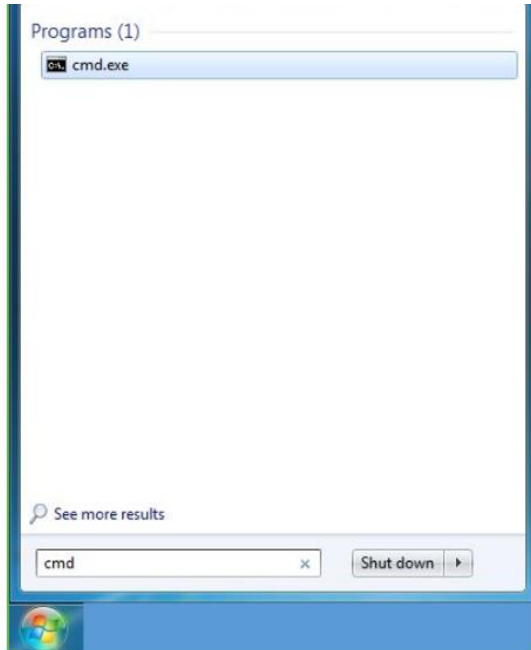
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

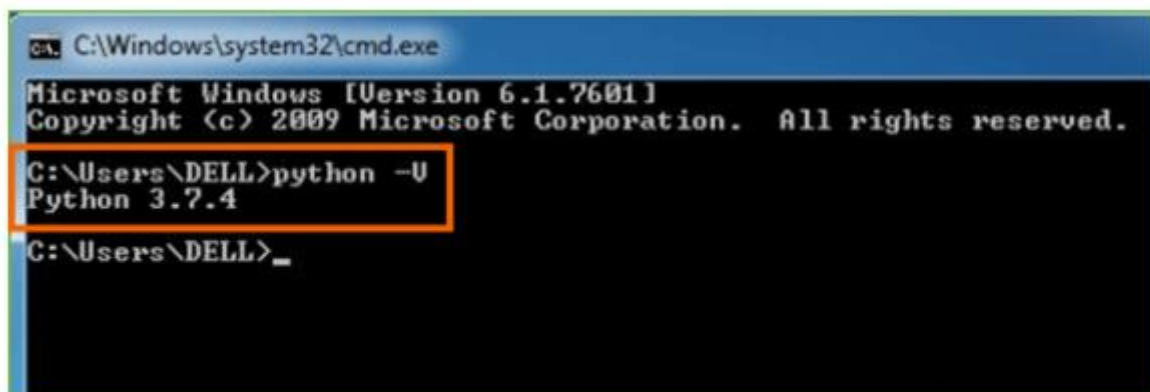
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python -V** and press Enter.



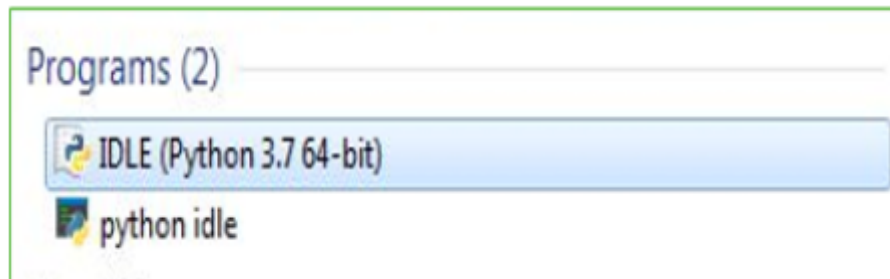
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

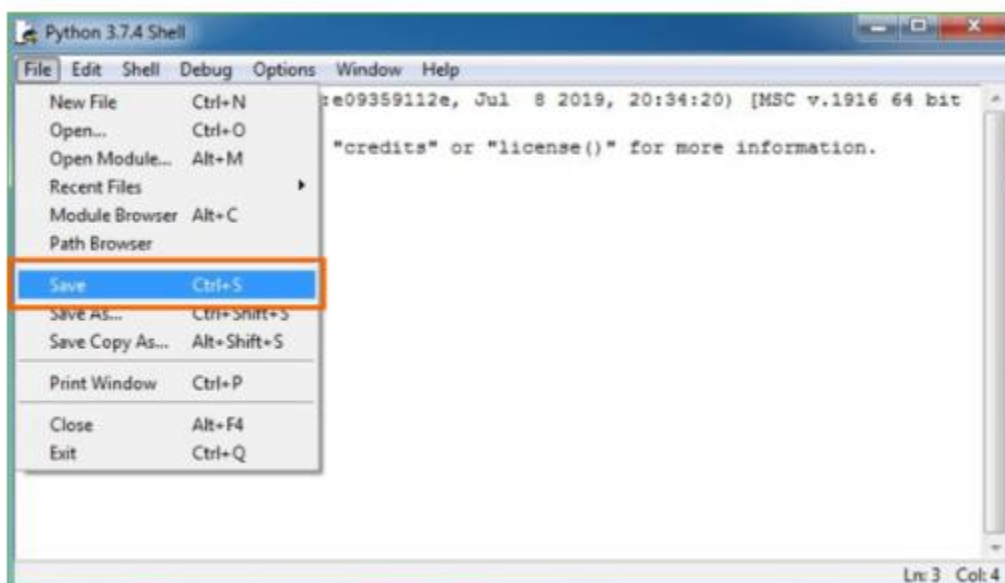
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

CHAPTER-VI

SYSTEM TESTING

6. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing :

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

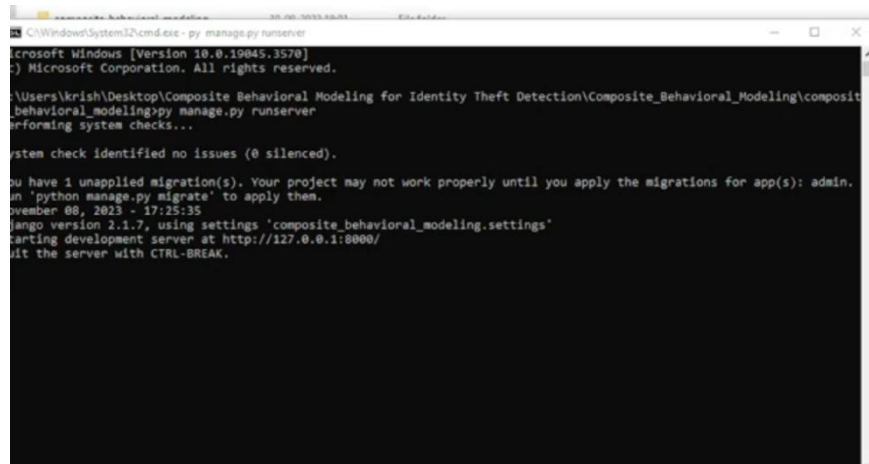
User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER-VII

SCREENSHOTS

7. SCREENSHOTS

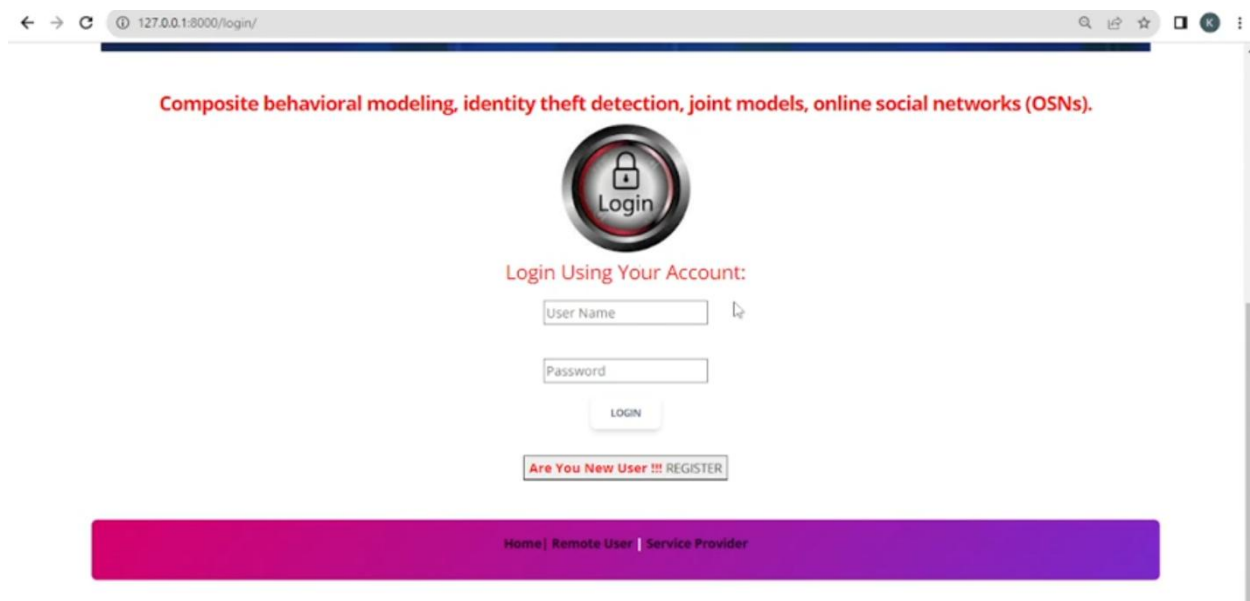


```
C:\Windows\System32\cmd.exe - py manage.py runserver
Microsoft Windows [Version 10.0.19045.3570]
(c) Microsoft Corporation. All rights reserved.

C:\Users\krish\Desktop\Composite Behavioral Modeling for Identity Theft Detection\Composite_Behavioral_Modeling\composite_behavioral_modeling>py manage.py runserver
Performing system checks...

System check identified no issues (0 silenced).

You have 1 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin.
Run 'python manage.py migrate' to apply them.
November 08, 2023 - 17:25:35
Django version 2.1.7, using settings 'composite_behavioral_modeling.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```



REGISTER

REGISTER YOUR DETAILS HERE !!!

Enter Username

User Name

Enter Password

Password

Enter EMail Id

Enter Email

Enter Address

Enter Address

Enter Gender

----Select Gender ----

Enter Mobile Number

Enter Mobile Number

Enter Country Name

Enter Country Name

Enter State Name

Enter State Name

Enter City Name

Enter City Name

REGISTER

Registered Status ::

Home | Remote User | Service Provider

Composite Behavior

Threat Detection in

PREDICT IDENTITY OF THEFT DETECTION | VIEW

Change your password

The password you just used was found in a data breach. Google Password Manager recommends changing your password now.

YOUR PROFILE DETAILS !!!

Username

user

Mobile Number

8798767898

Address

hyd

State

telangana

Email Id

user@gmail.com

Gender

Male

Country

india

City

hyderabad

127.0.0.1:8000/Predict_Theft_Status/

PREDICT IDENTITY OF THEFT DETECTION || VIEW YOUR PROFILE || LOGOUT

PREDICTION OF IDENTITY OF THEFT DETECTION !!!

ENTER DATASETS DETAILS HERE !!!

Enter Account_Id		Enter Trans_Id	
Enter Age		Enter Followers	
Enter NAME_CONTRACT_TYPE		Select GENDER	---Select---
Enter AMT_INCOME_TOTAL		Enter AMT_CREDIT	
Enter AMT_ANNUIITY		Enter AMT_GOODS_PRICE	
Enter NAME_INCOME_TYPE		Enter NAME_FAMILY_STATUS	

Predict

PREDICTION OF IDENTITY OF THEFT DETECTION !!!

ENTER DATASETS DETAILS HERE !!!

Enter Account_Id		Enter Trans_Id	
Enter Age		Enter Followers	
Enter NAME_CONTRACT_TYPE		Select GENDER	---Select---
Enter AMT_INCOME_TOTAL		Enter AMT_CREDIT	
Enter AMT_ANNUIITY		Enter AMT_GOODS_PRICE	
Enter NAME_INCOME_TYPE		Enter NAME_FAMILY_STATUS	

Predict

PREDICT IDENTITY OF THEFT DETECTION

Theft or Fraud Found

Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

Composite behavioral modeling, identity theft detection, joint models, online social networks (OSNs).



Login Service Provider:

User Name

Password

Login

User Login

Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

[Browse Datasets and Train & Test Data Sets](#) [View Trained and Tested Accuracy in Bar Chart](#) [View Trained and Tested Accuracy Results](#) [View Prediction Of Theft Detection Status](#) [View Theft Detection Status Ratio](#)
[Download Predicted Data Sets](#) [View Theft Detection Ratio Results](#) [View All Remote Users](#) [Logout](#)

Datasets Trained and Tested Results

Model Type	Accuracy
Extra Tree Classifier	51.62200282087447
SVM	51.198871650211565
Logistic Regression	51.57498824635637
Gradient Boosting Classifier	53.549600376116594

Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

[Browse Datasets and Train & Test Data Sets](#) [View Trained and Tested Accuracy in Bar Chart](#) [View Trained and Tested Accuracy Results](#) [View Prediction Of Theft Detection Status](#) [View Theft Detection Status Ratio](#)
[Download Predicted Data Sets](#) [View Theft Detection Ratio Results](#) [View All Remote Users](#) [Logout](#)



Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

[Browse Datasets and Train & Test Data Sets](#) [View Trained and Tested Accuracy in Bar Chart](#) [View Trained and Tested Accuracy Results](#) [View Prediction Of Theft Detection Status](#) [View Theft Detection Status Ratio](#)
[Download Predicted Data Sets](#) [View Theft Detection Ratio Results](#) [View All Remote Users](#) [Logout](#)



Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

[Browse Datasets and Train & Test Data Sets](#)
[View Trained and Tested Accuracy in Bar Chart](#)
[View Trained and Tested Accuracy Results](#)
[View Prediction Of Theft Detection Status](#)
[View Theft Detection Status Ratio](#)

[Download Predicted Data Sets](#)
[View Theft Detection Ratio Results](#)
[View All Remote Users](#)
[Logout](#)



View Theft Detect Prediction Type Details III

SENDER	AMT_INCOME_TOTAL	AMT_CREDIT	AMT_ANNUITY	AMT_GOODS_PRICE	NAME_INCOME_TYPE	NAME_FAMILY_STATUS	Prediction
	135000	312682.5	29686.5	297000	Working	Civil marriage	No Theft or Fraud Found
	99000	490495.5	27517.5	454500	State servant	Married	Theft or Fraud Found
	112500	157500	7875	157500	Working	Widow	No Theft or Fraud Found
	121500	1215000	48186	1215000	Pensioner	Married	Theft or Fraud Found
	202500	406597.5	24700.5	351000	Working	Single / not married	Theft or Fraud Found

Composite Behavioral Modeling for Identity Theft Detection in Online Social Networks

[Browse Datasets and Train & Test Data Sets](#) [View Trained and Tested Accuracy in Bar Chart](#) [View Trained and Tested Accuracy Results](#) [View Prediction Of Theft Detection Status](#) [View Theft Detection Status Ratio](#)
[Download Predicted Data Sets](#) [View Theft Detection Ratio Results](#) [View All Remote Users](#) [Logout](#)

VIEW ALL REMOTE USERS !!!

USER NAME	EMAIL	Gender	Address	Mob No	Country	State	City
Amaresh	Amaresh123@gmail.com	Male	#8928,4th Cross,Vijayanagar	9535866270	India	Karnataka	Bangalore
Manjunath	tnksmanju13@gmail.com	Male	#8928,4th Block,Vijayanagar	9535866270	India	Karnataka	Bangalore
user	user@gmail.com	Male	hyd	8798767898	india	telangana	hyderabad

CHAPTER-VIII

CONCLUSION

8. CONCLUSION

We investigate the feasibility of building a ladder from low-quality behavioral data to a high-performance behavioral model for user identification in OSNs. By deeply exploiting the complementary effect among OSN users' multidimensional behaviors, we propose a joint probabilistic generative model by integrating online and offline behaviors. When the designed joint model is applied to identity theft detection in OSNs, its comprehensive performance, in terms of the detection efficacy, response latency, and robustness, is validated by extensive evaluations on real-life OSN datasets. Particularly, the joint model significantly outperforms the existing fused model.

Our behavior-based method mainly aims at detecting identity thieves after the access control of the account is broken. Then, it is easy and promising to incorporate our method into traditional methods to solve the identity theft problem better.

REFERENCES

- [1] J. Onaolapo, E. Mariconti, and G. Stringhini, “What happens after you are pwnd: Understanding the use of leaked Webmail credentials in the wild,” in *Proc. Internet Meas. Conf.*, Nov. 2016, pp. 65–79.
- [2] A. Mohan, “A medical domain collaborative anomaly detection framework for identifying medical identity theft,” in *Proc. Int. Conf. Collaboration Technol. Syst. (CTS)*, May 2014, pp. 428–435.
- [3] Y.-A. de Montjoye, L. Radaelli, V. K. Singh, and A. S. Pentland, “Unique in the shopping mall: On the reidentifiability of credit card metadata,” *Science*, vol. 347, no. 6221, pp. 536–539, Jan. 2015.
- [4] P. Hyman, “Cybercrime: It’s serious, but exactly how serious?” *Commun. ACM*, vol. 56, no. 3, pp. 18–20, Mar. 2013.
- [5] L. Bilge, T. Strufe, D. Balzarotti, and E. Kirda, “All your contacts are belong to us: Automated identity theft attacks on social networks,” in *Proc. 18th Int. Conf. World Wide Web (WWW)*, 2009, pp. 551–560.
- [6] J. Lynch, “Identity theft in cyberspace: Crime control methods and their effectiveness in combating phishing attacks,” *Berkeley Technol. Law J.*, vol. 20, no. 1, pp. 259–300, 2005.
- [7] M. Egele, G. Stringhini, C. Kruegel, and G. Vigna, “Towards detecting compromised accounts on social networks,” *IEEE Trans. Dependable Secure Comput.*, vol. 14, no. 4, pp. 447–460, Jul. 2017.
- [8] T. C. Pratt, K. Holtfreter, and M. D. Reisig, “Routine online activity and Internet fraud targeting: Extending the generality of routine activity theory,” *J. Res. Crime Delinquency*, vol. 47, no. 3, pp. 267–296, Aug. 2010.
- [9] K. Thomas, C. Grier, J. Ma, V. Paxson, and D. Song, “Design and evaluation of a real-time URL spam filtering service,” in *Proc. IEEE Symp. Secur. Privacy*, May 2011, pp. 447–462.
- [10] H. Li et al., “Bimodal distribution and co-bursting in review spam detection,” in *Proc. 26th Int. Conf. World Wide Web*, Apr. 2017, pp. 1063–1072.
- [11] A. M. Marshall and B. C. Tompsett, “Identity theft in an online world,” *Comput. Law Secur. Rev.*, vol. 21, no. 2, pp. 128–137, Jan. 2005.
- [12] B. Schneier, “Two-factor authentication: Too little, too late,” *Commun. ACM*, vol. 48, no. 4, p. 136, Apr. 2005.

- [13] M. V. Ruiz-Blondet, Z. Jin, and S. Laszlo, "CEREBRE: A novel method for very high accuracy event-related potential biometric identification," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 7, pp. 1618–1629, Jul. 2016.
- [14] R. D. Labati, A. Genovese, E. Muñoz, V. Piuri, F. Scotti, and G. Sforza, "Biometric recognition in automated border control: A survey," *ACM Comput. Surv.*, vol. 49, no. 2, p. 24, 2016.
- [15] B. A. Rajoub and R. Zwigelaar, "Thermal facial analysis for deception detection," *IEEE Trans. Inf. Forensics Security*, vol. 9, no. 6, pp. 1015–1023, Jun. 2014.
- [16] M. M. Waldrop, "How to hack the hackers: The human side of cybercrime," *Nature*, vol. 533, no. 7602, pp. 164–167, May 2016.
- [17] C. Wang, B. Yang, J. Cui, and C. Wang, "Fusing behavioral projection models for identity theft detection in online social networks," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 4, pp. 637–648, Aug. 2019.
- [18] C. Shen, Y. Li, Y. Chen, X. Guan, and R. A. Maxion, "Performance analysis of multi-motion sensor behavior for active smartphone authentication," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 48–62, Jan. 2018.
- [19] C. Wang and H. Zhu, "Representing fine-grained co-occurrences for behavior-based fraud detection in online payment services," *IEEE Trans. Dependable Secure Comput.*, early access, May 4, 2020, doi: 10.1109/TDSC.2020.2991872.
- [20] H. Zheng et al., "Smoke screener or straight shooter: Detecting elite sybil attacks in user-review social networks," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2018, pp. 259–300.
- [21] R. T. Mercuri, "Scoping identity theft," *Commun. ACM*, vol. 49, no. 5, pp. 17–21, May 2006.
- [22] G. Stringhini, P. Mourlanne, G. Jacob, M. Egele, C. Kruegel, and G. Vigna, "EVILCOHORT: Detecting communities of malicious accounts on online services," in *Proc. USENIX Secur.*, 2015, pp. 563–578.
- [23] Q. Cao, X. Yang, J. Yu, and C. Palow, "Uncovering large groups of active malicious accounts in online social networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 477–488.
- [24] G. Stringhini, C. Kruegel, and G. Vigna, "Detecting spammers on social networks," in *Proc. 26th Annu. Comput. Secur. Appl. Conf. (ACSAC)*, 2010, pp. 1–9.

[25] Y. Yao, B. Viswanath, J. Cryan, H. Zheng, and B. Y. Zhao, “Automated crowdturfing attacks and defenses in online review systems,” in Proc. ACM SIGSAC Conf. Comput. Commun. Secur., Dallas, TX, USA, Oct. 2017, pp. 1143–1158.